Customer Shopping Behaviour Analysis

Python (pandas) → MySQL → SQL analysis → Power BI dashboard
Dataset: customer_shopping_behavior.csv, 3,900 customers, 18 columns. Files covered: Customer_Shopping_Behaviour_.ipynb, Customer_Behaviuor.sql, customer_behavior_dashboard.pbix.

1. Project workflow

The project follows four stages: (1) load and clean the raw CSV in Python, (2) engineer new features, (3) push the cleaned table into a MySQL database, and (4) answer business questions with SQL and visualise them in Power BI.

2. What happened in the Python notebook

Step 1 - Load the data. `pd.read_csv("customer_shopping_behavior.csv")` loads 3,900 rows and 18 columns (Customer ID, Age, Gender, Item Purchased, Category, Purchase Amount (USD), Location, Size, Color, Season, Review Rating, Subscription Status, Shipping Type, Discount Applied, Promo Code Used, Previous Purchases, Payment Method, Frequency of Purchases).

Step 2 - Explore. `df.head()`, `df.info()` and `df.describe(include="all")` showed: ages 18-70 (mean 44.1), purchase amount USD 20-100 (mean 59.76), 25 distinct items in 4 categories, 50 locations, 2 genders (2,652 male), and 73% of customers not subscribed (2,847 of 3,900).

Step 3 - Check missing values. `df.isnull().sum()` found that only **Review Rating** had nulls: 37 of 3,900 rows (info() showed 3,863 non-null).

Step 4 - Impute missing ratings. Nulls were filled with the **median rating of the same Category** (`groupby("Category").transform(lambda x: x.fillna(x.median()))`). The median is robust to outliers, and grouping by category keeps the fill realistic. Afterwards all columns had 0 nulls and the mean rating stayed ~3.75.

Step 5 - Standardise column names. Columns were lower-cased and spaces replaced by underscores; `purchase_amount_(usd)` was renamed to `purchase_amount` so it is safe to use in SQL.

Step 6 - Create age_group. `pd.qcut(age, q=4)` split customers into four equal-sized quartile bands: Young Adult (18-31), Adult (32-44), Middle-aged (45-57), Senior (58-70).

Step 7 - Create purchase_frequency_days. The text column Frequency of Purchases was mapped to days: Weekly 7, Fortnightly/Bi-Weekly 14, Monthly 30, Quarterly/Every 3 Months 90, Annually 365. This gives a numeric measure that can be averaged and compared.

Step 8 - Remove redundant column. `(df.discount_applied == df.promo_code_used).all()` returned True: the two columns are identical in every row, so **promo_code_used** was dropped.

Step 9 - Connect to MySQL. pandas, SQLAlchemy and PyMySQL were used. The password is entered with `getpass` (not stored in the notebook), a SQLAlchemy `URL.create(mysql+pymysql, root@localhost:3306/mydatabase)` engine was built, and a test connection printed "MySQL connected successfully!".

Step 10 - Upload the table. `df.to_sql("mytable", con=engine, if_exists="replace", index=False)` wrote the cleaned dataframe (19 columns incl. age_group and purchase_frequency_days) into table **mytable**.

3. SQL analysis (MySQL, table mytable)

The .sql file starts with `show databases; use mydatabase; select * from mytable limit 20;` to confirm the upload. Each business question below is answered with the query from your file, followed by its result. Results were reproduced on the same cleaned data, so they should match your MySQL output.

Q1. Total revenue by gender

```
select gender, sum(purchase_amount)
from mytable group by gender;
```

Groups sales by gender and sums spend. Male customers generate about 2.1x the revenue of female customers (they are 68% of customers).

gender	sum(purchase_amount)
Female	75191
Male	157890

Q2. Discount users spending at or above average

```
select customer_id, purchase_amount from mytable
where discount_applied = "Yes"
and purchase_amount >= (select avg(purchase_amount) from mytable);
```

A subquery computes the overall average (USD 59.76); the outer query keeps discounted purchases at or above it. It returns **839 customers** (average spend USD 79.79). Full list is in MySQL; only the count is shown here.

customers returned	avg purchase (USD)
839.0	79.79

Q3. Top 5 products by average review rating

```
select item_purchased as Products,
 round(avg(review_rating),2) as "Product Review Ratings"
from mytable group by item_purchased
order by avg(review_rating) desc limit 5;
```

Ratings are averaged per product, sorted high to low. Differences are small (3.78-3.86), so all top products are rated similarly.

Product	Rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.8
Skirt	3.78

Q4. Average purchase: Standard vs Express shipping

```
select shipping_type, avg(purchase_amount) as "Average Purchase Amounts"
from mytable where shipping_type in ("Express","Standard")
group by shipping_type;
```

Express buyers spend about USD 2 more on average than Standard buyers - a small gap.

shipping_type	avg_purchase
Express	60.48
Standard	58.46

Q5. Do subscribers spend more?

```
select subscription_status, count(customer_id) as total_customers,
 round(avg(purchase_amount),2) as average_spend,
 sum(purchase_amount) as total_revenue
from mytable group by subscription_status
order by average_spend desc, total_revenue desc;
```

No. Subscribers spend essentially the same per purchase (USD 59.49 vs 59.87). Non-subscribers bring more total revenue simply because there are more of them.

subscription_status	customers	avg_spend	total_revenue
No	2847	59.87	170436
Yes	1053	59.49	62645

Q6. Revenue share by subscription category

```
select subscription_status, round(sum(purchase_amount),2) as total_revenue,
 round(sum((purchase_amount)*100/(select sum(purchase_amount) from mytable)),2)
 as percentage_revenue
from mytable group by subscription_status;
```

Each row is expressed as a percentage of the grand total (USD 233,081). Non-subscribers 73.1%, subscribers 26.9%.

subscription_status	total_revenue	percentage_revenue
No	170436	73.12
Yes	62645	26.88

Q7. Top 5 products by discount rate

```
select item_purchased as "Top 5 Products",
 round(sum(discount_applied="Yes")*100/count(*),2) as Discount_rate
from mytable group by item_purchased
order by Discount_rate desc limit 5;
-- alternative: sum(case when discount_applied="Yes" then 1 else 0 end)
```

Two equivalent versions are in your file: a boolean sum, and a CASE WHEN sum. Both give the same result. About half of all Hat, Sneakers and Coat purchases used a discount.

Product	discount_rate
Hat	50.0
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

Q8. Customer segments (New / Returning / Loyal)

```
with customer_type as (
 select customer_id, previous_purchases,
 case when previous_purchases = 1 then "New"
      when previous_purchases between 2 and 10 then "Returning"
      else "Loyal" end as customer_segment
 from mytable)
select customer_segment, count(*) as "Number of Customers"
from customer_type group by customer_segment;
```

A CTE labels each customer by number of previous purchases (1 = New, 2-10 = Returning, 11+ = Loyal). 80% of customers are Loyal; only 83 are New.

customer_segment	Number of Customers
Loyal	3116
Returning	701
New	83

Q9. Top 3 products per category

```
with item_count as (
 select category, item_purchased, count(customer_id) as Total_Orders,
  row_number() over (partition by category
   order by count(customer_id) desc) as item_rank
 from mytable group by category, item_purchased)
select item_rank, category, item_purchased, Total_Orders
from item_count where item_rank <= 3;
```

A window function ranks products inside each category. Outerwear has only two products (Jacket, Coat). Note: ROW_NUMBER breaks ties arbitrarily (e.g. Sunglasses and Belt both have 161 orders); use RANK() or DENSE_RANK() if ties should show equally.

item_rank	category	item_purchased	Total_Orders
1	Accessories	Jewelry	171
2	Accessories	Sunglasses	161
3	Accessories	Belt	161
1	Clothing	Pants	171
2	Clothing	Blouse	171
3	Clothing	Shirt	169
1	Footwear	Sandals	160
2	Footwear	Shoes	150
3	Footwear	Sneakers	145
1	Outerwear	Jacket	163
2	Outerwear	Coat	161

Q10. Are repeat buyers likely to subscribe?

```
select subscription_status, count(customer_id) as repeat_buyers
from mytable where previous_purchases > 5
group by subscription_status;
```

Of 3,476 repeat buyers (more than 5 previous purchases), 958 (27.6%) are subscribed - almost the same as the 27.0% across all customers. Repeat buying does **not** make subscription noticeably more likely.

subscription_status	repeat_buyers
No	2518
Yes	958

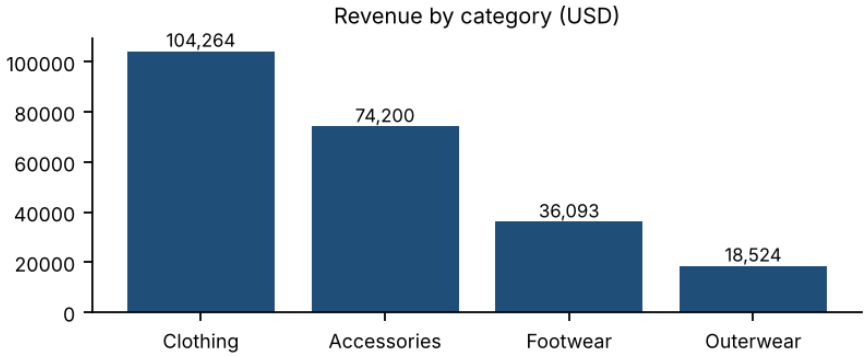
Q11. Revenue contribution by age group

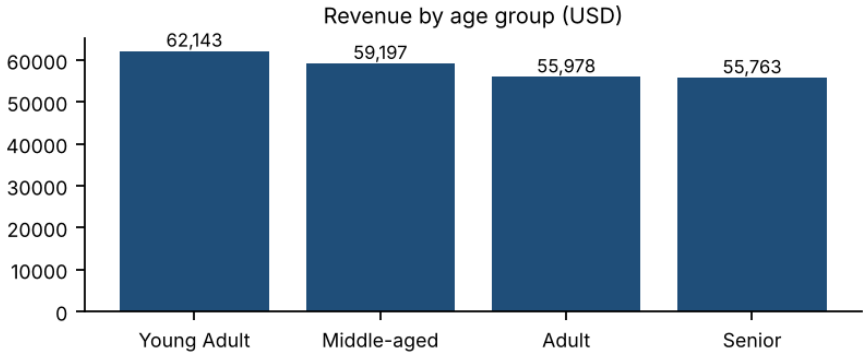
```
select sum(purchase_amount) as Total_Revenue, age_group
from mytable group by age_group order by Total_Revenue desc;

select round(sum(purchase_amount)*100/(select sum(purchase_amount)
 from mytable),2) as Revenue_percentage, age_group
from mytable group by age_group order by Revenue_percentage desc;
```

Two queries: absolute revenue, then percentage share. Because age groups are quartiles (equal sizes), contributions are similar; Young Adults lead slightly at 26.7%.

age_group	revenue	pct
Young Adult	62143	26.66
Middle-aged	59197	25.4
Adult	55978	24.02
Senior	55763	23.92

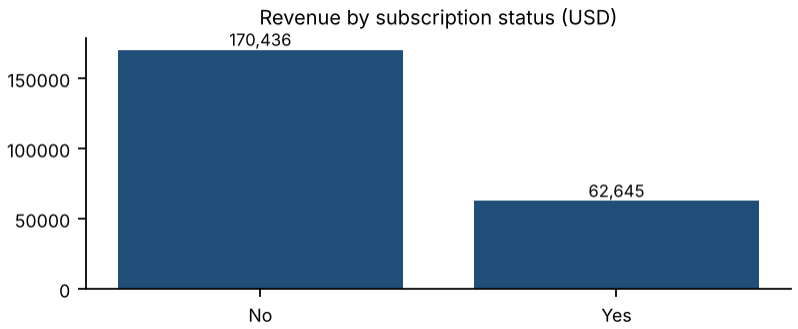




4. Power BI dashboard

Your file **customer_behavior_dashboard.pbix** contains a one-page dashboard built on the MySQL table (named *public customer* in the model). I read its layout. It contains:
• **KPI cards:** Number of Customers, Average Purchase Amount, Average Review Rating.
• **Donut chart:** customers by subscription status.
• **Column charts:** total purchase amount by category, and customer count by category.
• **Bar charts:** total purchase amount by age group, and customer count by age group.
• **Slicers (filters):** subscription status, gender, category, shipping type.

Values the KPI cards should show on the full data: 3,900 customers, average purchase USD 59.76, average rating 3.75; total revenue USD 233,081.



Insights the dashboard supports

• Clothing is the largest category by revenue (USD 104,264, 45%), followed by Accessories (USD 74,200).
• Male customers drive about two-thirds of revenue.
• Subscribers are 27% of customers and 27% of revenue, with the same average spend as non-subscribers - subscription is not yet increasing spend.
• Age groups contribute almost equally (23.9%-26.7% each).
• Most customers (80%) are Loyal (11+ previous purchases): growth lies in acquiring new customers.
• Discounts are used by about 43% of purchases; some products (Hat, Sneakers, Coat) reach roughly 50%.

How to rebuild or extend it in Power BI Desktop

1. Get Data → MySQL database (localhost, mydatabase) → select *mytable* → Load.
2. Create measures (Modeling → New measure): Number of Customers = DISTINCTCOUNT(customer_id); Average Purchase Amount = AVERAGE(purchase_amount); Average Review Rating = AVERAGE(review_rating). Note: the donut and bar charts in the file use Sum of customer_id for counts, which adds ID numbers rather than counting customers; switch these to Count of customer_id or the Number of Customers measure.
3. Add visuals: cards for the three measures, donut for subscription status, columns for category, bars for age group.
4. Add slicers for subscription_status, gender, category and shipping_type.
5. Optional extras: a customer-segment chart using a calculated column (New/Returning/Loyal as in Q8), top products by rating, and discount rate by product.

Note: I cannot create or edit Power BI files in this environment, so the dashboard section documents the .pbix you provided rather than a new build.